



UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

► **Curso: Fundamentos de Programación**

► **Docente: Américo Chulluncuy Reynoso**

2025-3

Sesión 14:

Programación Orientada a Objetos

Contenido

1. Programación orientada a objetos (POO)

2. Herencia

Tipos de herencia

Implementación de herencia

3. Introducción al polimorfismo

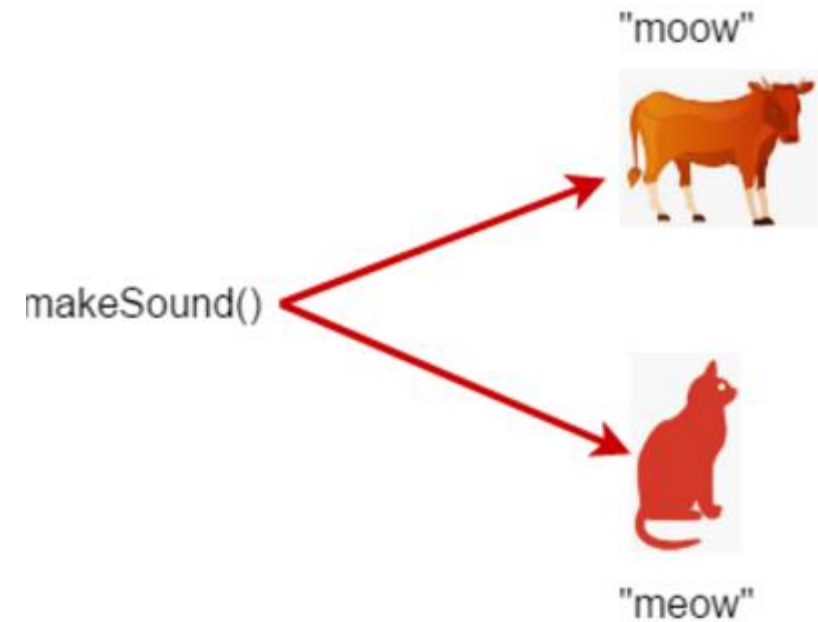
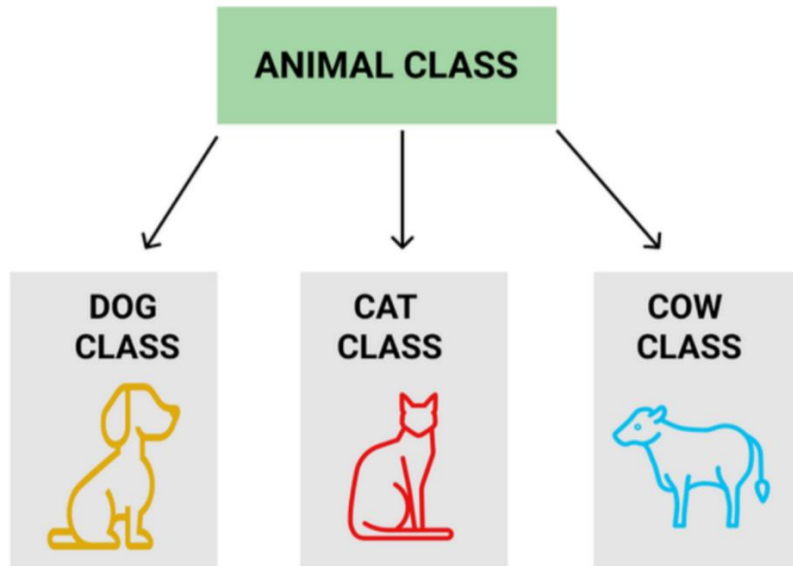
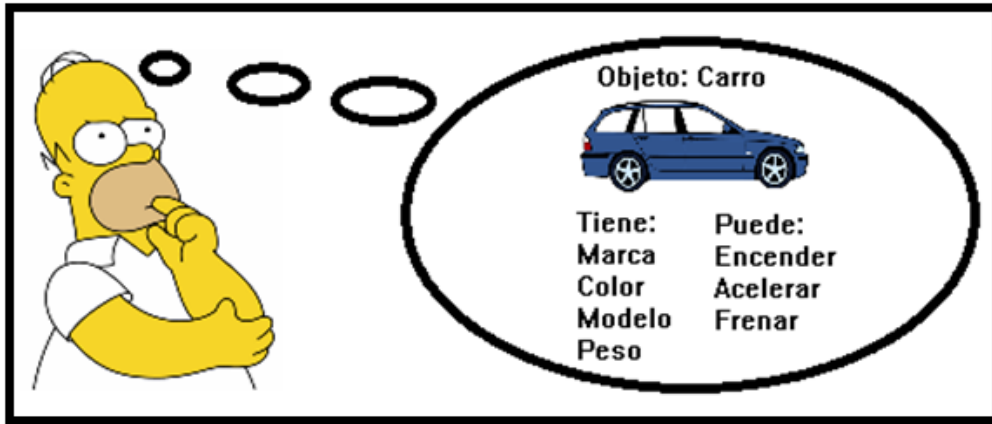
Conceptos básicos en POO

- **Objeto:** es una instancia específica de una clase. Cada objeto ocupa su propia área de memoria y tiene un conjunto único de valores que representan su estado.
- **Clase:** define las propiedades (atributos) y el comportamiento (métodos) comunes a un grupo de objetos. Actúa como un tipo de dato definido por el usuario, a partir del cual se pueden crear múltiples instancias (objetos).

Conceptos básicos en POO

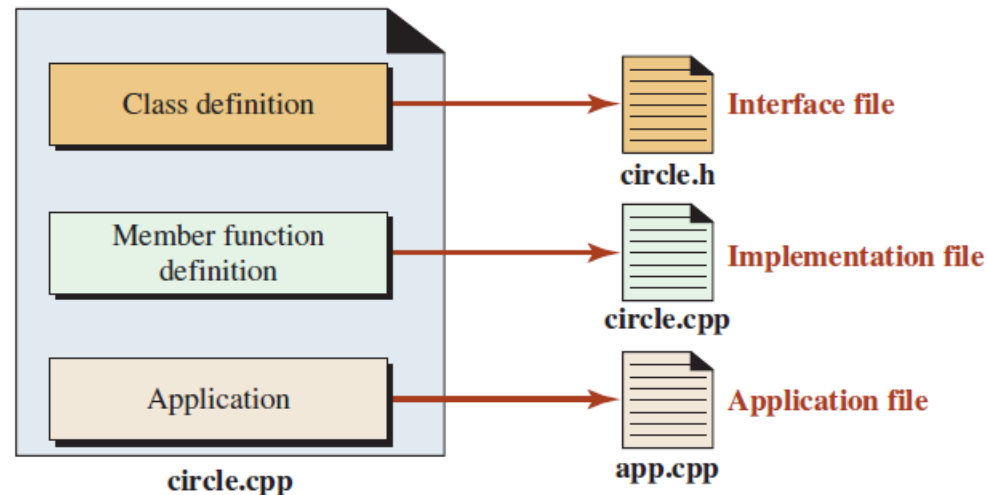
- **Encapsulación:** es el principio de **agrupar datos** (atributos) **y funciones** (métodos) que operan sobre esos datos en una sola unidad: la clase. Esto **permite ocultar los detalles internos de implementación**, haciendo que los datos sean accesibles únicamente a través de métodos.
- **Herencia:** Una **clase derivada hereda los atributos y métodos de una clase base**. Esto **facilita la reutilización de código** y la **creación de jerarquías de clases que comparten comportamiento común**, mientras permiten a las clases derivadas extender o modificar la funcionalidad heredada.
- **Polimorfismo** es la capacidad de un elemento del programa, como una función o un operador, de **adoptar diferentes comportamientos dependiendo de la clase a la que pertenezca el objeto que lo invoca**. Esto se logra típicamente mediante sobrecarga (overloading) o sobrescritura (overriding)

Principios de la POO



1. Programación Orientada a Objetos

- El paso de la programación procedimental a la Orientada a Objetos requiere implementar algunos cambios en la forma en que compilamos y ejecutamos programas.
- Recordemos, cuando se utiliza C++ como lenguaje de programación orientado a objetos, normalmente hay tres secciones de código: **definición de clase, definición de funciones miembros y aplicación**. C++ nos permite crear tres archivos separados, uno para cada sección.

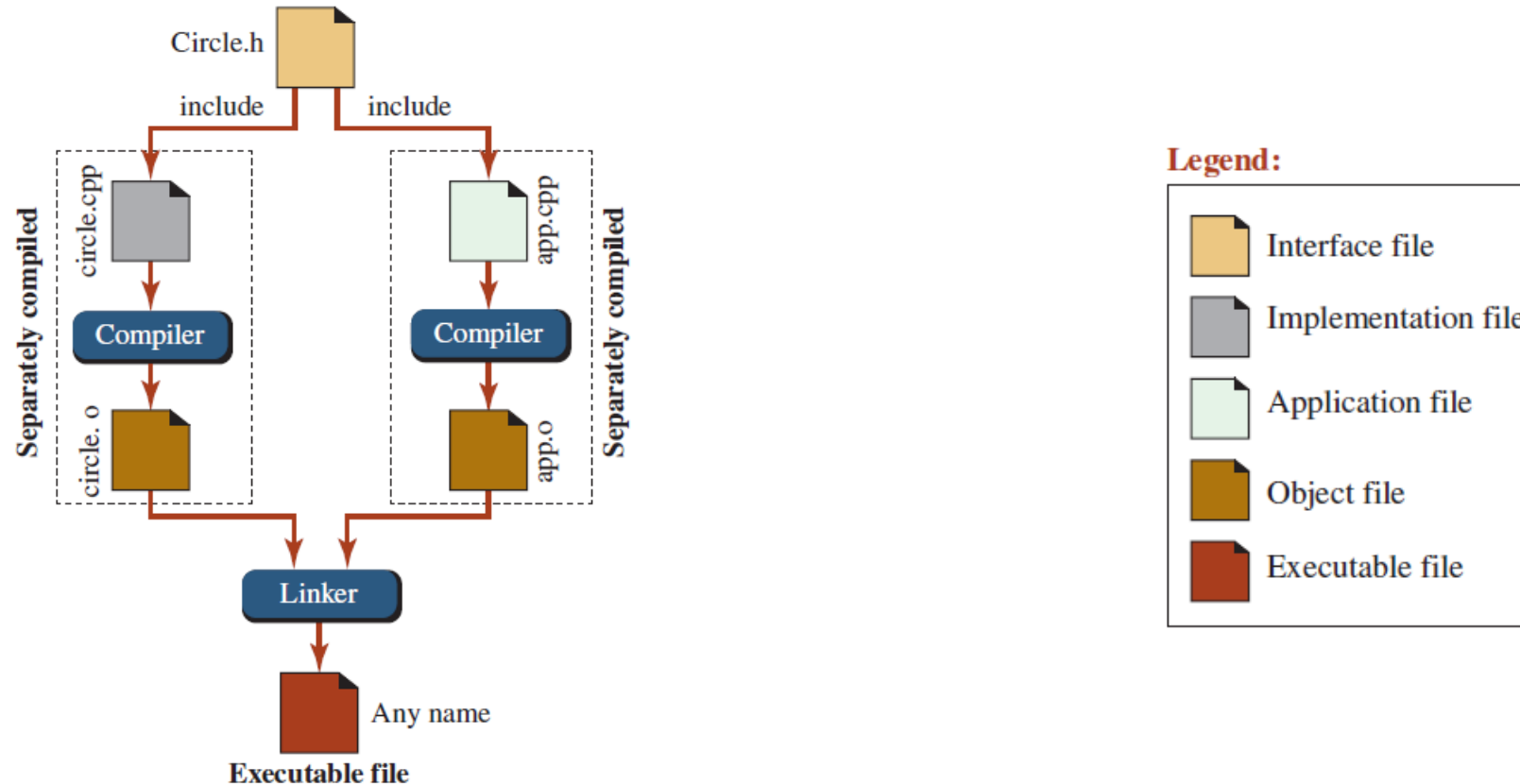


Los 3 archivos creados en C++ para una Clase

- **Archivo de interfaz:** contiene la definición de clase (declaraciones de miembros de datos y de funciones de miembros). Normalmente tiene el nombre de la clase con una extensión h, como circulo.h. La letra h lo designa como un archivo de encabezado.
- **Archivo de implementación:** contiene la definición de funciones miembro. Normalmente tiene el nombre de la clase con extensión .cpp, como en circulo.cpp.
- **Archivo de aplicación:** incluye la función principal que se utiliza para crear instancias de objetos y permitir que cada objeto realice operaciones sobre sí mismo. Normalmente tiene extensión .cpp. El nombre del archivo lo elige el usuario.

Compilando los 3 archivos

- Una vez creados los 3 archivos separados, debemos compilarlos (compilación separada) para crear un archivo ejecutable.
- Si bien el proceso es el mismo, el nombre de los comandos de compilación y/o archivos creados pueden diferir en diferentes entornos.



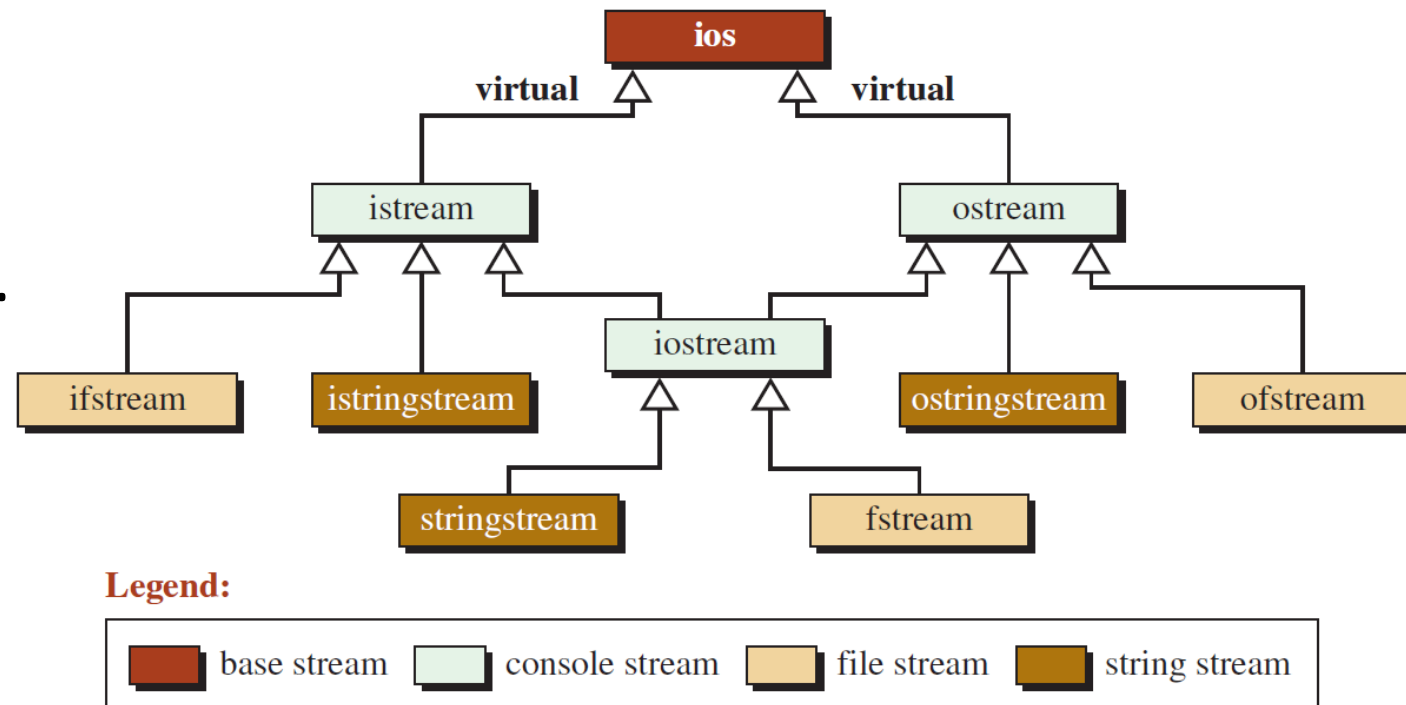
Ejemplo del proceso de compilación

- a. El **archivo de interfaz** se crea y contiene solo la definición de clase, este archivo **debe incluirse en el archivo de implementación y en el archivo de aplicación**. En nuestra clase Rectangulo, podemos llamar a este archivo rectangulo.h.
- b. El **archivo de implementación** se crea con el archivo de interfaz incluyendo una directiva de inclusión. Si deseamos que solo se realice la compilación usamos : `c++ -c rectangulo.cpp` Si la compilación es exitosa, tenemos un archivo objeto con la extensión o. `rectangulo.o`.
- c. Se crea el **archivo de la aplicación** en el que el **archivo de interfaz también se agrega al principio del archivo usando la directiva de inclusión**. Este archivo también se compila usando el siguiente comando: `c++ -c main.cpp`. Si la compilación es exitosa, tenemos un archivo objeto con la extensión o. `main.o`.
- d. Luego enlazamos los dos archivos objeto con la opción `-o` para crear un archivo ejecutable: `c++ -o aplicacion rectangulo.o main.o`.
- e. El resultado es un archivo ejecutable que se puede ejecutar como se muestra a continuación: `c++ aplicacion`

Ejemplo 1: programaOO muestra el procedimiento descrito.

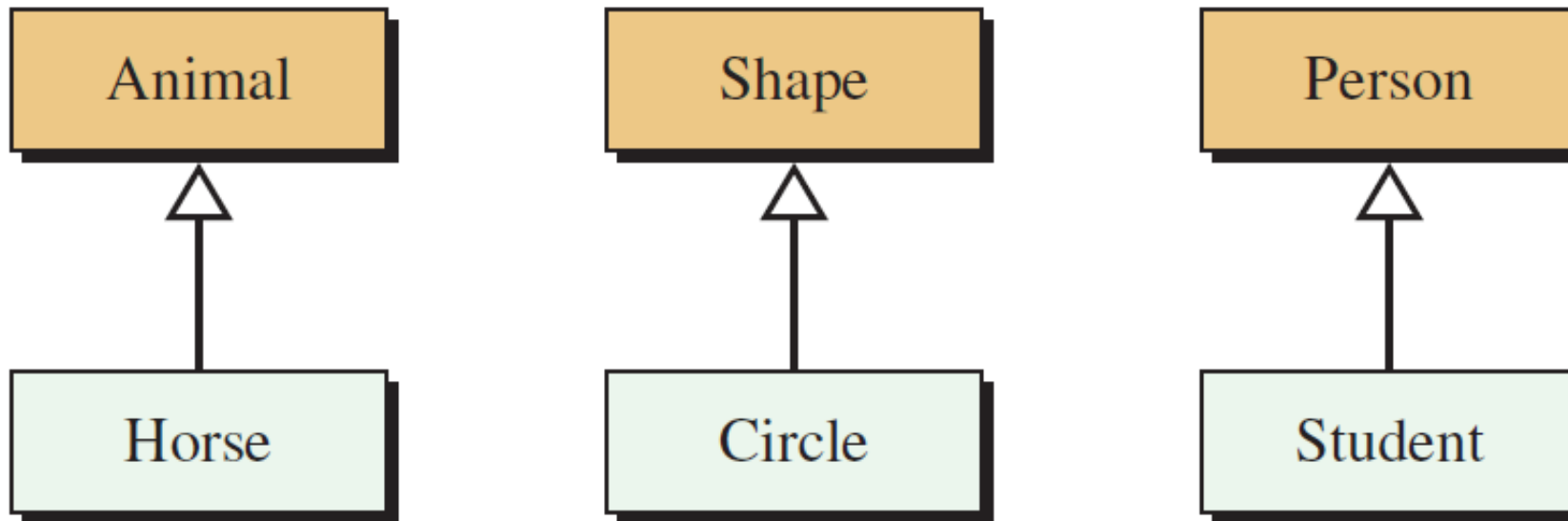
2. Herencia

- Las clases nos permiten (además de crear tipos de datos abstractos) crear nuevas clases a partir de clases existentes, fomentando la reutilización del código.
- En POO, las clases no se utilizan de forma aislada. Un programa normalmente utiliza varias clases con diferentes relaciones entre ellas (jerarquía de clases).



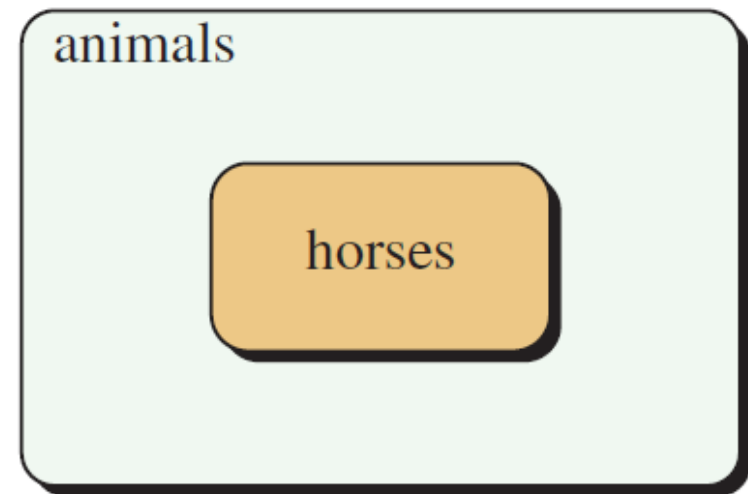
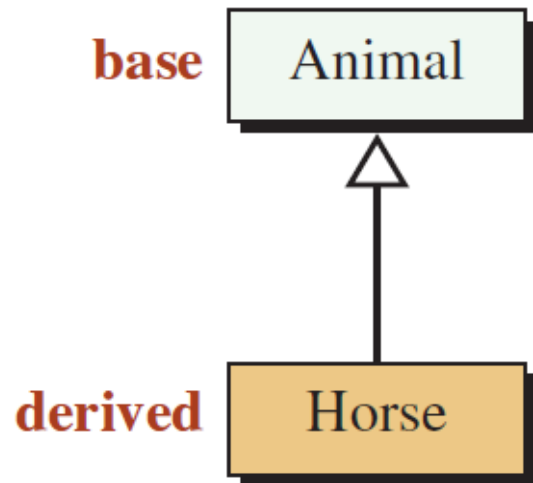
Herencia

- En POO, la herencia **deriva un concepto más específico de uno más general**. Por ejemplo, el concepto de animal en biología es más general que el concepto de caballo.
- Hay una **relación** “es-un(a)” desde lo más específico a lo más general. Todos los caballos son animales, pero no todos los animales son caballos.



Herencia

- La clase más general se llama **clase base** (superclase) y una clase más específica se llama **clase derivada** (subclase).
- Una **clase derivada extiende su clase base**, es decir, la clase derivada hereda todos los miembros de datos y funciones miembros de la clase base (**excepto** constructores, destructores y operadores de asignación que deben redefinirse) y puede crear nuevos miembros de datos y funciones.



Tipos de Herencia

- Para **crear una clase derivada** a partir de una clase base, tenemos tres opciones en C++: herencia privada (**por defecto**), herencia protegida y herencia pública.
- Para **mostrar el tipo de herencia que queremos usar**, insertamos dos puntos después de la clase, seguido de una de la palabra clave (privada, protegida o pública):

```
class D : public B
```

```
{
```

```
...
```

```
};
```

```
class D : protected B
```

```
{
```

```
...
```

```
};
```

```
class D : private B
```

```
{
```

```
...
```

```
};
```

Tipos de Herencia, **B** es la clase base y **D** es la clase derivada

Implementación de herencia

- **Ejemplo 2** Supongamos que necesitamos implementar dos clases que llamaremos Suma y Resta.

Cada clase tiene como atributo **valor1**, **valor2** y **resultado**.

Los métodos a definir son **cargar1** (que inicializa el atributo valor1), **cargar2** (que inicializa el atributo valor2), **operar** (que en el caso de la clase "Suma" suma los dos atributos y en el caso de la clase "Resta" hace la diferencia entre valor1 y valor2) y

otro método **mostrarResultado**

Ejercicio

- Implementar una clase Persona que tenga como atributos el nombre y la edad. Definir como responsabilidades un método que cargue los datos personales y otro que los imprima.

Plantear una segunda clase Empleado que herede de la clase Persona. Añadir un atributo sueldo y los métodos de cargar el sueldo e imprimir su sueldo.

Definir un objeto de la clase Persona y llamar a sus métodos. También crear un objeto de la clase Empleado y llamar a sus métodos.

3. Introducción al Polimorfismo

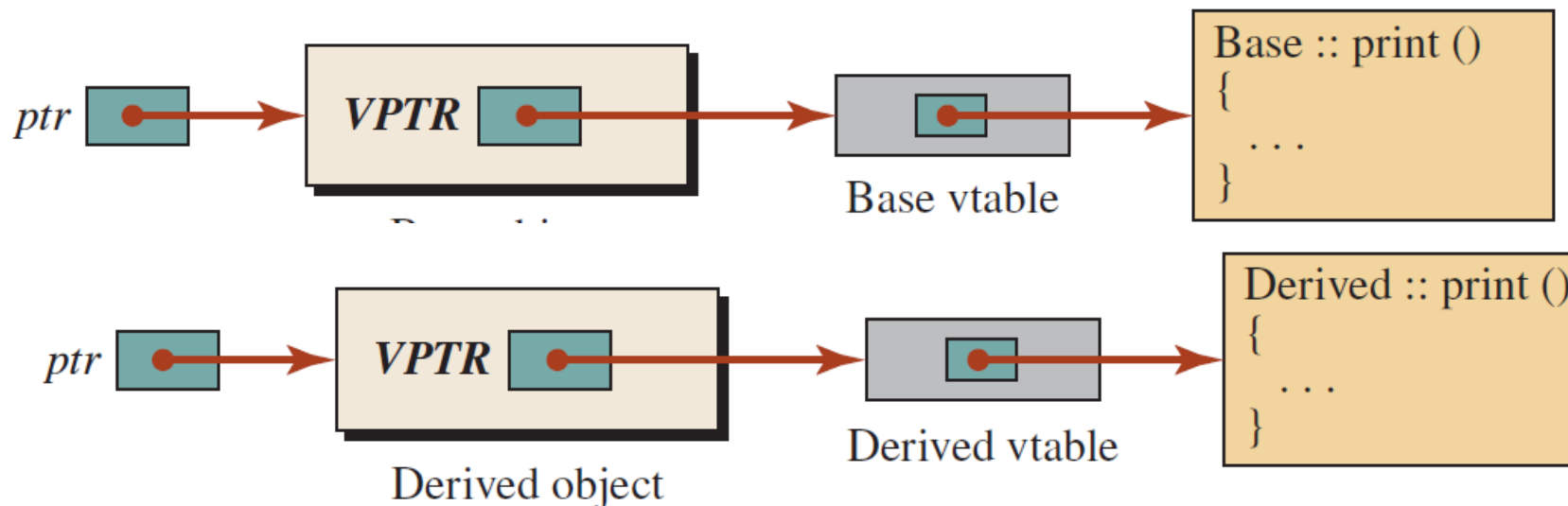
- El polimorfismo, nos da la **capacidad de escribir varias versiones de una función en diferentes clases**. Al llamar la función se ejecuta la versión apropiada para el objeto al que se hace referencia.
- Por ejemplo, podemos llamar a la función `printArea` para imprimir el área de un triángulo o el área de un rectángulo.
- El mismo concepto ocurre en nuestro lenguaje cotidiano. Por ejemplo, el significado del verbo “abrir”, está determinado por el contexto: abrir un libro, abrir un frasco, abrir una puerta, etc

Condiciones para el polimorfismo en la herencia

- **Punteros o referencias:** puede apuntar a la clase base; entonces podemos dejar que el puntero apunte a cualquier objeto en la jerarquía.
- **Objetos intercambiables:** es un objeto en una jerarquía de herencia
- **Funciones virtuales:** se modifican con la palabra clave virtual. Por ejemplo, podemos tener una función imprimir en todas las clases, todas con el mismo nombre, pero cada una imprimiendo de manera diferente.
- El [ejemplo 3](#) muestra el uso de las dos primeras condiciones.
- El [ejemplo 4](#) muestra el uso de las 3 condiciones, logrando alcanzar el polimorfismo

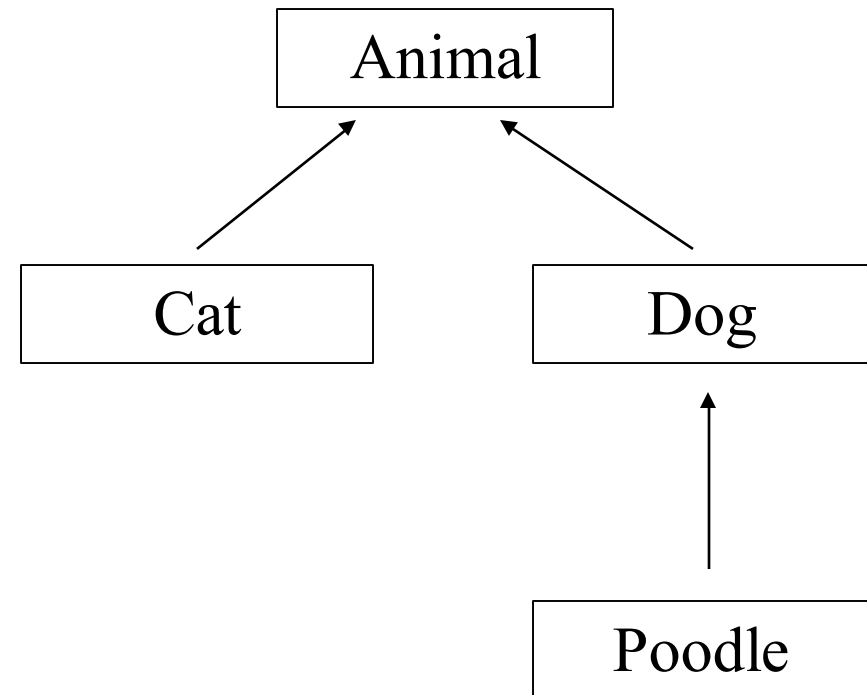
Funciones virtuales y polimorfismo

- Cuando una clase declara al menos una función como `virtual`, el compilador genera internamente una estructura llamada **tabla virtual** o **vtable**, que **contiene punteros a las implementaciones reales de las funciones virtuales de esa clase**.
- Cada objeto instanciado a partir de una clase que tiene funciones virtuales contiene internamente un **puntero oculto** (*VPTR*) que apunta a la vtable de la clase a la que realmente pertenece el objeto.
- Gracias a ese puntero, cuando se llama a una función virtual a través de un puntero o referencia a la clase base, el programa puede decidir en tiempo de ejecución qué versión de la función ejecutar. Esto es lo que permite el **polimorfismo dinámico**.



Resumen: Polimorfismo

- **Código polimórfico:** Código que se comporta de manera diferente cuando actúa sobre objetos de diferentes tipos
- **Función miembro virtual:** el mecanismo de C++ para lograr polimorfismo
- Ejemplo: Considere la jerarquía Animal, Cat, Dog donde cada clase tiene su propia versión de la función miembro id()



Ejemplo: sin polimorfismo

```
class Animal{  
public: void id(){cout << "animal";}  
};
```

```
class Cat : public Animal{  
    public: void id(){cout << "cat";}  
};
```

```
class Dog : public Animal{  
    public: void id(){cout << "dog";}  
};
```

- Considere la colección de diferentes objetos de animales
`Animal *pA[] = {new Animal, new Dog, new Cat};`
- El código
`for(int k=0; k<3; k++)
 pA[k]->id();`
- Imprime `animal animal animal` ignorando las versiones específicas de `id()` en `Dog` y `Cat`
- **El código anterior no es polimórfico**: se comporta de la misma manera, aunque `Animal`, `Dog` y `Cat` tengan diferentes tipos y diferentes funciones miembro `id()`.

Ejemplo: código polimórfico

```
class Animal{  
public: virtual void id() override {cout << "animal";}  
};  
  
class Cat : public Animal{  
    public: virtual void id() override {cout << "cat";}  
};  
  
class Dog : public Animal{  
    public: virtual void id() override {cout << "dog";}  
};
```

- En el ejemplo sin polimorfismo, en la expresión `pA[k] -> id()` el compilador solo ve el tipo de puntero `pA[k]`, que es un puntero a `Animal`.
- El compilador no ve el tipo de objeto real al que apunta, que puede ser `Animal`, `Dog` o `Cat`
- Declarar una **función virtual** hará que el compilador verifique el tipo de cada objeto para ver si define una versión más específica de la función virtual.

Ejercicios Resueltos

Ejemplos/ejercicios básicos:

https://www.w3resource.com/cpp-exercises/oop/index.php#google_vignette

Ejemplos Clases, Constructores, Herencia (simple, múltiple), Polimorfismo

https://github.com/andresWeitzel/Ejercicios_Resueltos_Cpp/tree/master/Bloque%2015%20-%20POO